

Declarative Attention: A Model That Says Which Parts to Reread

Kabui, Charles

2026-09-29

 Read at ToKnow.ai



Researchers at KAIST AI and Google DeepMind have published a way to let a long-context language model decide, as it writes, which parts of its input it needs to reread. Chat models normally reread the entire conversation to produce each word, even when the answer sits in one paragraph. Their method, [Declarative Attention](#), has the model tag its own reasoning with one of three scopes: `<global>` for the full input, `<focus>` for named sections, or `<local>`

for its recent output. The engine watches for those tags, parses them like tool calls, and skips the rest of the memory read. With no retraining, tested on two off-the-shelf models across 15 long-context tasks, it cut the context tokens read per reply by 52% on Gemma-4-31B and 31% on Qwen-3.6-27B.

Accuracy fell 1.27% on Gemma-4-31B and 2.75% on Qwen-3.6-27B, and the loss shrinks as models get bigger. Because the method is mostly a prompt plus a smarter engine, an [independent PyTorch implementation](#) is already available. The authors estimate, from a hardware roofline rather than a real stopwatch, that decode time drops to about 0.71x of normal on Gemma. The models also take roughly a third more decode steps, and on a few tasks the tokens read go up.

Most sparse-attention systems use a separate scorer to guess which tokens matter. Here the model is its own router, using a channel it already produces, its chain of thought. That makes an attention decision readable, and the savings grow with model size.

Read More: [Attention Residuals](#) is another drop-in attention fix that works without retraining, while [TriAttention](#) attacks the same long-context cost bill by shrinking the memory itself.

Sources:

- [Language Models Can Control Their Own Attention \(arXiv\)](#)
- [Full paper: results, protocol adherence and limitations \(arXiv HTML\)](#)
- [Language Models Can Control Their Own Attention \(Hugging Face Papers\)](#)
- [Unofficial PyTorch implementation of Declarative Attention](#)

***Disclaimer:** For information only. Accuracy or completeness not guaranteed. Illegal use prohibited. Not professional advice or solicitation. **Read more:** [/terms-of-service](#)*